



บทที่ 5

การแปลงข้อมูล

- ✓ 5.1 แนะนำการแปลงข้อมูล
- ✓ 5.2 Encoding
- ✓ 5.3 Min-Max Normalization
- ✓ 5.4 Standardization
- ✓ 5.5 Box-Cox
- ✓ 5.6 Discretization
- ✓ 5.7 แบบฝึกหัด

5.1 แนะนำการแปลงข้อมูล

- การแปลงข้อมูล (Data Transformation) คือ การดัดแปลงข้อมูลให้อยู่ในรูปแบบที่เหมาะสมต่อการนำไปใช้สร้างโมเดล
- ตัวอย่างเทคนิคของการแปลงข้อมูล มีดังนี้
 - Encoding
 - Min-Max Normalization
 - Standardization
 - Box-Cox
 - Discretization

5.2 Encoding

- เป็นวิธีที่สะดวก
- เหมาะกับตัวแปรเชิงกลุ่ม (Categorical) หรือ Nominal/Ordinal
- มีหลายเทคนิค เช่น
 - One-Hot Encoding เหมาะกับตัวแปร Nominal
 - Ordinal Encoding เหมาะกับตัวแปร Ordinal
 - Label Encoding เหมาะกับตัวแปรตาม Nominal/Ordinalบางกรณีใช้กับ ตัวแปรต้นที่มี 2 ค่า (Binomial) เช่น เพศชาย เพศหญิง

5.2.1 One-Hot Encoding

One-Hot Encoding เป็นวิธีการเข้ารหัสข้อมูล ซึ่งเหมาะกับตัวแปรแบบ Nominal โดยแปลงกลุ่มข้อมูลให้เป็นค่าตัวเลข 0 หรือ 1 ในคอลัมน์ใหม่

หลักการ One-Hot Encoding

- ระบุกลุ่มข้อมูล ของตัวแปร (เช่น ชาย, บัณฑิต, บุคคล)
- สร้างคอลัมน์ใหม่ ตามจำนวนกลุ่ม
- พิจารณาแต่ละแถว ถ้าข้อมูลตรงกับกลุ่มไหนให้ค่าเป็น 1 ส่วนกลุ่มที่เหลือ ให้ค่าเป็น 0

item	age	sex	education_level	occupation	total_spend	พนักงานชาย	พนักงานบัณฑิต	พนักงานบุคคล
1	25	หญิง	ปริญญาตรี	พนักงานชาย	4500	1	0	0
2	27	ชาย	ปริญญาโท	พนักงานบัณฑิต	6200	0	1	0
3	23	ชาย	ปริญญาตรี	พนักงานชาย	5000	1	0	0
4	35	หญิง	ปริญญาเอก	พนักงานบุคคล	8000	0	0	1
5	41	ชาย	ปริญญาโท	พนักงานบัณฑิต	120000	0	1	0
6	32	หญิง	ปริญญาตรี	พนักงานบัณฑิต	3500	0	1	0
7	26	หญิง	ปริญญาโท	พนักงานชาย	11000	1	0	0
8	22	ชาย	ปริญญาตรี	พนักงานบุคคล	8750	0	0	1
9	28	ชาย	ปริญญาตรี	พนักงานบุคคล	15000	0	0	1
10	34	ชาย	ปริญญาตรี	พนักงานชาย	12000	1	0	0

โค้ดไพทอนสำหรับ One-Hot Encoding

```
1 #Import libraries
2 import pandas as pd
3 from sklearn.preprocessing import OneHotEncoder
4
5 #Load data
6 x = pd.read_csv('data/customer3.csv')
7
8 #Encode
9 trans = OneHotEncoder(categories=[['พนักงานขาย', 'พนักงานบัญชี', 'พนักงานบุคคล']])
10 trans.fit(x[['occupation']])
11 x_array = trans.transform(x[['occupation']]).toarray().astype(int)
12
13 #Concat data
14 x_df = pd.DataFrame(x_array, columns=trans.categories[0])
15 x = pd.concat([x, x_df], axis=1)
16
17 #Drop a column
18 x = x.drop(columns=['occupation'])
19 x
```

item	age	sex	education_level	total_spend	พนักงานขาย	พนักงานบัญชี	พนักงานบุคคล
1	25	หญิง	ปริญญาตรี	4500	1	0	0
2	27	ชาย	ปริญญาโท	6200	0	1	0
3	23	ชาย	ปริญญาตรี	5000	1	0	0
4	35	หญิง	ปริญญาเอก	8000	0	0	1
5	41	ชาย	ปริญญาโท	120000	0	1	0
6	32	หญิง	ปริญญาตรี	3500	0	1	0
7	26	หญิง	ปริญญาโท	11000	1	0	0
8	22	ชาย	ปริญญาตรี	8750	0	0	1
9	28	ชาย	ปริญญาตรี	15000	0	0	1
10	34	ชาย	ปริญญาตรี	12000	1	0	0

5.2.2 Ordinal Encoding

Ordinal Encoding เป็นวิธีการเข้ารหัสข้อมูล ซึ่งเหมาะกับตัวแปรแบบ Ordinal โดยแปลงกลุ่มข้อมูลให้เป็น ค่าตัวเลขที่มีลำดับ (Ordering) เช่น 0, 1, 2, ... หรือ 1, 2, 3, ...

หลักการ Ordinal Encoding

- ระบุลำดับของกลุ่มข้อมูลของตัวแปร เช่น ตริ < โท < เอก
- แทนค่ากลุ่มข้อมูลเดิมด้วยลำดับของตัวเลข
เช่น ถ้าแถวนี้อยู่ในกลุ่ม “ตริ” → แทนด้วย 0
ถ้าแถวนี้อยู่ในกลุ่ม “โท” → แทนด้วย 1
ถ้าแถวนี้อยู่ในกลุ่ม “เอก” → แทนด้วย 2

item	age	sex	education_level	occupation	total_spend
1	25	หญิง	0	พนักงานขาย	4500
2	27	ชาย	1	พนักงานบัญชี	6200
3	23	ชาย	0	พนักงานขาย	5000
4	35	หญิง	2	พนักงานบุคคล	8000
5	41	ชาย	1	พนักงานบัญชี	120000
6	32	หญิง	0	พนักงานบัญชี	3500
7	26	หญิง	1	พนักงานขาย	11000
8	22	ชาย	0	พนักงานบุคคล	8750
9	28	ชาย	0	พนักงานบุคคล	15000
10	34	ชาย	0	พนักงานขาย	12000

โค้ดไพทอนสำหรับ Ordinal Encoding

```
1 #Import libraries
2 import pandas as pd
3 from sklearn.preprocessing import OrdinalEncoder
4
5 #Load data
6 x = pd.read_csv('data/customer3.csv')
7
8 #Encode
9 trans = OrdinalEncoder(
10     categories=[['ปริญญาตรี', 'ปริญญาโท', 'ปริญญาเอก']])
11 trans.fit(x[['education_level']])
12 x['education_level']=trans.transform(x[['education_level']]).astype(int)
13 x
```

item	age	sex	education_level	occupation	total_spend
1	25	หญิง	0	พนักงานขาย	4500
2	27	ชาย	1	พนักงานบัญชี	6200
3	23	ชาย	0	พนักงานขาย	5000
4	35	หญิง	2	พนักงานบุคคล	8000
5	41	ชาย	1	พนักงานบัญชี	120000
6	32	หญิง	0	พนักงานบัญชี	3500
7	26	หญิง	1	พนักงานขาย	11000
8	22	ชาย	0	พนักงานบุคคล	8750
9	28	ชาย	0	พนักงานบุคคล	15000
10	34	ชาย	0	พนักงานขาย	12000

5.2.3 Label Encoding

Label Encoding เป็นวิธีการเข้ารหัสข้อมูล ซึ่งเหมาะกับตัวแปรตามแบบ Nominal และ Ordinal โดยแปลงกลุ่มข้อมูลให้เป็น ตัวเลข โดยปกติจะแทนตามลำดับ Unicode หรือ ตามหลักภาษา (แต่ไม่ได้หมายความว่าลำดับที่มาก่อนมีความสำคัญกว่า)

หลักการ Label Encoding

- ระบุกลุ่มข้อมูล ของตัวแปร
- แทน กลุ่มข้อมูล ด้วย ค่าตามลำดับ Unicode หรือ ตามหลักภาษา
เช่น “Apple” → แทนด้วย 0
“Banana” → แทนด้วย 1
“Lemon” → แทนด้วย 2
“Mango” → แทนด้วย 3
“Orange” → แทนด้วย 4

item	weight	sweetness	sourness	color	fruit_name
1	150	7	2	Red	0 Apple
2	180	8	1	Red	0 Apple
3	120	6	3	Orange	4 Orange
4	130	5	4	Orange	4 Orange
5	90	9	1	Yellow	1 Banana
6	100	8	1	Yellow	1 Banana
7	250	5	8	Green	2 Lemon
8	270	4	9	Green	2 Lemon
9	160	7	3	Yellow-Green	3 Mango
10	170	8	2	Yellow-Green	3 Mango

โค้ดไพทอนสำหรับ Label Encoding

```
1 #Import libraries
2 import pandas as pd
3 from sklearn.preprocessing import LabelEncoder
4
5 #Load data
6 x = pd.read_csv('data/fruits.csv')
7
8 #Encode (เรียงตามลำดับ Unicode)
9 trans = LabelEncoder()
10 trans.fit(x['fruit_name'])
11 x['fruit_name']=trans.transform(x['fruit_name'])
12
13 #Show outputs
14 print(f'Class: {trans.classes_}')
15 x
```

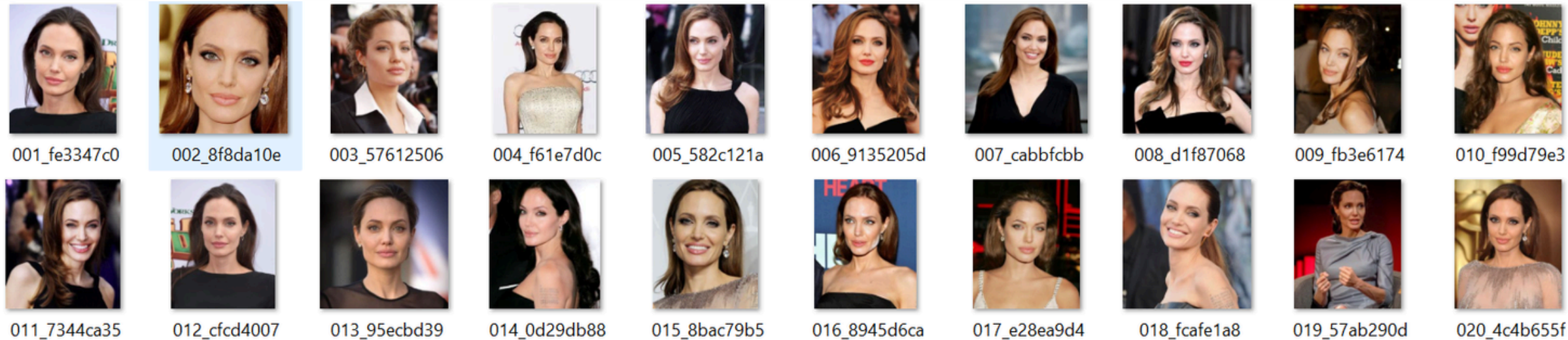
Class: ['Apple' 'Banana' 'Lemon' 'Mango' 'Orange']

item	weight	sweetness	sourness	color	fruit_name
1	150	7	2	Red	0
2	180	8	1	Red	0
3	120	6	3	Orange	4
4	130	5	4	Orange	4
5	90	9	1	Yellow	1
6	100	8	1	Yellow	1
7	250	5	8	Green	2
8	270	4	9	Green	2
9	160	7	3	Yellow-Green	3
10	170	8	2	Yellow-Green	3

Name

- Angelina Jolie
- Brad Pitt
- Denzel Washington
- Jennifer Lawrence
- Johnny Depp
- Leonardo DiCaprio
- Megan Fox
- Natalie Portman
- Nicole Kidman
- Will Smith

Angelina Jolie



```
1 import os
2 import numpy as np
3 import tensorflow as tf
4
5 def load_data(data_dir, img_size=(224, 224)):
6     images, labels = [], []
7
8     # สร้าง label map
9     label_map = {name: idx for idx, name in enumerate(os.listdir(data_dir))}
10
11     for label in os.listdir(data_dir):
12         folder_path = os.path.join(data_dir, label)
13
14         # ข้ามถ้าไม่ใช่โฟลเดอร์
15         if not os.path.isdir(folder_path):
16             continue
17
18         for file in os.listdir(folder_path):
19             img_path = os.path.join(folder_path, file)
20
```

```
21 # อ่านรูปด้วย TensorFlow
22 img = tf.io.read_file(img_path)
23 img = tf.image.decode_image(img, channels=3, expand_animations=False)
24 img = tf.image.resize(img, img_size)
25 img = img / 255.0 # Normalize
26
27 images.append(img.numpy()) # Convert to NumPy
28 labels.append(label_map[label])
29
30 return np.array(images), np.array(labels), label_map
31
32 # Load data
33 data_dir = 'data/celeb'
34 X, y, label_map = load_data(data_dir)
35
36 print("X shape:", X.shape)
37 print("y shape:", y.shape)
38 print("label_map:", label_map)
39 print("labels example:", y[:30])
```

X shape: (200, 224, 224, 3)

y shape: (200,)

label_map: {'Angelina Jolie': 0, 'Brad Pitt': 1, 'Denzel Washington': 2, 'Jennifer Lawrence': 3, 'Johnny Depp': 4, 'Leonardo DiCaprio': 5, ... , 'Will Smith': 9}

labels example: [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1]

5.3 Min-Max Normalization

- เป็นเทคนิคการแปลงข้อมูลให้อยู่ในช่วงค่าที่กำหนด เช่น $[0, 1]$ หรือ $[-1, 1]$ โดยใช้ ค่าต่ำสุด (Min) และ ค่าสูงสุด (Max) ของตัวแปรนั้น ๆ
- เหมาะกับ ML ที่เป็น Distance-based ที่ต้องการลดผลกระทบจากสเกลของตัวแปรที่แตกต่างกัน เช่น KNN, K-Means, Hierarchical Clustering
- เหมาะกับ ML ที่มีการใช้ Activation Function แบบมีช่วงค่าจำกัด เช่น Neural Networks ที่ต้องการใช้ Logistic Sigmoid: ช่วง $[0, 1]$, Tanh: ช่วง $[-1, 1]$
- เหมาะกับ ML ที่ใช้การ Optimization ด้วย Gradient Descent เช่น Neural Networks เนื่องจากการทำ Scaling ให้ค่ามีขนาดเล็กและอยู่ในสเกลใกล้เคียงกัน จะช่วยให้การ Train โมเดล เสถียร และ Converge ได้เร็วขึ้น

5.3.1 การคำนวณสำหรับ Min-Max Normalization

สูตรการคำนวณ

$$\hat{x} = \frac{x - \min}{\max - \min}(\text{new_max} - \text{new_min}) + \text{new_min}$$

เมื่อ

$\hat{x}$ = ค่าที่แปลงแล้ว

x = ค่าที่ต้องการแปลง

$\min$ = ค่าต่ำสุดในช่วงเก่า

$\max$ = ค่าสูงสุดในช่วงเก่า

new_min = ค่าต่ำสุดในช่วงใหม่

new_max = ค่าสูงสุดในช่วงใหม่

ตัวอย่างการคำนวณ

จากข้อมูลต่อไปนี้ จงแปลงข้อมูลให้อยู่ในช่วง [0, 1]

$x = [25, 27, 23, 35, 41, 32, 26, 22, 28, 34]$

วิธีทำ

จาก
$$\hat{x} = \frac{x - \min}{\max - \min} (\text{new_max} - \text{new_min}) + \text{new_min}$$

$$\min = 22$$

$$\max = 41$$

$$\text{new_min} = 0$$

$$\text{new_max} = 1$$

เมื่อ $x = 25$ จะได้

$$\hat{x} = \frac{25 - 22}{41 - 22} (1 - 0) + 0$$

$$\hat{x} = 0.16$$

เมื่อ $x = 27$ จะได้

$$\hat{x} = \frac{27 - 22}{41 - 22} (1 - 0) + 0 \quad \dots$$

$$\hat{x} = 0.26$$

เมื่อ $x = 34$ จะได้

$$\hat{x} = \frac{34 - 22}{41 - 22} (1 - 0) + 0$$

$$\hat{x} = 0.63$$

5.3.2 โค้ดไพทอนสำหรับ Min-Max Normalization

```
1 #Import libraries
2 import pandas as pd
3 from sklearn.preprocessing import MinMaxScaler
4
5 #Load data
6 x = pd.read_csv('data/customer3.csv')
7
8 #Normalize data
9 trans=MinMaxScaler((0,1))
10 trans.fit(x[['age']])
11 x[['age']]=trans.transform(x[['age']]).round(2)
12 x
```

item	age	sex	education_level	occupation	total_spend
1	0.16	หญิง	ปริญญาตรี	พนักงานขาย	4500
2	0.26	ชาย	ปริญญาโท	พนักงานบัญชี	6200
3	0.05	ชาย	ปริญญาตรี	พนักงานขาย	5000
4	0.68	หญิง	ปริญญาเอก	พนักงานบุคคล	8000
5	1.00	ชาย	ปริญญาโท	พนักงานบัญชี	120000
6	0.53	หญิง	ปริญญาตรี	พนักงานบัญชี	3500
7	0.21	หญิง	ปริญญาโท	พนักงานขาย	11000
8	0.00	ชาย	ปริญญาตรี	พนักงานบุคคล	8750
9	0.32	ชาย	ปริญญาตรี	พนักงานบุคคล	15000
10	0.63	ชาย	ปริญญาตรี	พนักงานขาย	12000

5.4 Standardization

- เป็นเทคนิคการแปลงข้อมูลให้มี ค่าเฉลี่ย (μ) = 0 และ ส่วนเบี่ยงเบนมาตรฐาน (σ) = 1
 - ข้อมูลหลังการแปลงจะอยู่ในช่วงประมาณ $[-3, +3]$ (แต่ก็เป็นไปได้ว่าจะมีค่านอกขอบเขตนี้)
 - เหมาะกับ ML ที่เป็น Distance-based ที่ต้องการลดผลกระทบจากสเกลของตัวแปรที่แตกต่างกัน เช่น KNN, K-Means, Hierarchical Clustering
 - เหมาะกับ ML ที่ต้องการพิจารณาความสำคัญของ ตัวแปร ด้วยค่า Coefficients เช่น Linear Regression
 - เหมาะกับ ML ที่ใช้การ Optimization ด้วย Gradient Descent เช่น Neural Networks เนื่องจากการทำ Scaling ให้ค่ามีขนาดเล็กและอยู่ในสเกลใกล้เคียงกัน จะช่วยให้การ Train โมเดล เสถียร และ Converge ได้เร็วขึ้น
- (ในกรณีข้อมูลภาพ Normalization จะเหมาะกว่า โดยแปลง [0 – 255] เป็น [0 – 1] เนื่องจาก Normalization ไม่ได้ไปเปลี่ยนความหมายของภาพ เช่น ความสว่างของภาพ)**

Technique	จำกัดช่วงของข้อมูล	ลดผลกระทบจากสเกลของตัวแปรที่แตกต่างกัน	การตีความตัวแปรจาก Coefficients	การ Train Model ด้วย Gradient Descent
Standardization	น้อยมาก	ปานกลาง	ตีความได้	เร็ว
Normalization	มาก	มาก	ไม่ได้	ปานกลาง

5.4.1 การคำนวณสำหรับ Standardization

สูตรการคำนวณ

$$z = \frac{x - \mu}{\sigma}$$

หรือ

$$z = \frac{x - \bar{x}}{s}$$

เมื่อ

z = ค่า z - score

x = ข้อมูลแต่ละจุด

μ = ค่าเฉลี่ยของประชากร (Population)

σ = ส่วนเบี่ยงเบนมาตรฐานของประชากร

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$$

เมื่อ

z = ค่า z - score

x = ข้อมูลแต่ละจุด

$\bar{x}$ = ค่าเฉลี่ยของตัวอย่าง (Samples)

s = ส่วนเบี่ยงเบนมาตรฐานของตัวอย่าง

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Standardization ทำให้ค่าเฉลี่ยเป็น 0 จึงทำให้ข้อมูลมีจุดกึ่งกลางใหม่ที่สมมาตรในเชิงตำแหน่ง แต่รูปร่าง (shape) ของข้อมูล เช่น ความเอียง (skewness) จะไม่เปลี่ยนแปลง

ตัวอย่างการคำนวณ

จากข้อมูลต่อไปนี้ จงแปลงค่าด้วยวิธี Standardization

$$x = [25, 27, 23, 35, 41, 32, 26, 22, 28, 34]$$

วิธีทำ

จาก $z = \frac{x - \mu}{\sigma}$

$$\bar{x} = 29.3$$

$$\sigma = 5.73$$

เมื่อ $x = 25$ จะได้

$$\hat{x} = \frac{25 - 29.3}{5.73}$$

$$\hat{x} = -0.75$$

เมื่อ $x = 27$ จะได้

$$\hat{x} = \frac{27 - 29.3}{5.73}$$

$$\hat{x} = -0.40$$

.....

เมื่อ $x = 34$ จะได้

$$\hat{x} = \frac{34 - 29.3}{5.73}$$

$$\hat{x} = 0.82$$

5.4.2 โค้ดไพทอนสำหรับ Standardization

```
1 #Import libraries
2 import pandas as pd
3 from sklearn.preprocessing import StandardScaler
4
5 #Load data
6 x = pd.read_csv('data/customer3.csv')
7
8 #Standardize data
9 trans=StandardScaler()
10 trans.fit(x[['age']])
11 x[['age']]=trans.transform(x[['age']]).round(2)
12 x
```

item	age	sex	education_level	occupation	total_spend
1	-0.75	หญิง	ปริญญาตรี	พนักงานขาย	4500
2	-0.40	ชาย	ปริญญาโท	พนักงานบัญชี	6200
3	-1.10	ชาย	ปริญญาตรี	พนักงานขาย	5000
4	1.00	หญิง	ปริญญาเอก	พนักงานบุคคล	8000
5	2.04	ชาย	ปริญญาโท	พนักงานบัญชี	120000
6	0.47	หญิง	ปริญญาตรี	พนักงานบัญชี	3500
7	-0.58	หญิง	ปริญญาโท	พนักงานขาย	11000
8	-1.27	ชาย	ปริญญาตรี	พนักงานบุคคล	8750
9	-0.23	ชาย	ปริญญาตรี	พนักงานบุคคล	15000
10	0.82	ชาย	ปริญญาตรี	พนักงานขาย	12000

5.5 Box-Cox

- เป็นเทคนิคการแปลงข้อมูลด้วย Power Transformation (และ Log เมื่อ $\lambda = 0$)
ค่า λ (lambda) ถูกใช้เพื่อควบคุมระดับความโค้ง หรือ ความเบ้ของข้อมูล
- ทำให้ข้อมูลมีทั้ง หดตัวลง (Compress) หรือ ขยายตัวออก (Stretch)
- การหดตัว ช่วยลดความเบ้ (Skewness) ทำให้ลดผลกระทบของ Outliers ได้ดีขึ้น
- การขยายตัว ช่วยให้ข้อมูลมีการกระจายตัวมากขึ้น มีโอกาสแบ่งแยก Class หรือ ทำนายค่าต่อเนื่องได้ง่ายขึ้น
- ใช้ได้เฉพาะข้อมูลที่เป็นบวก ($x > 0$)

5.5.1 การคำนวณสำหรับ Box-Cox

สูตรการคำนวณ

$$\hat{x} = \begin{cases} \frac{x^\lambda - 1}{\lambda}, & \lambda \neq 0 \\ \ln(x), & \lambda = 0 \end{cases}$$

เมื่อ

$\hat{x}$ = ค่าที่แปลงแล้ว

x = ค่าที่ต้องการแปลง

λ = ค่าคงที่ ได้มาจากการประมาณค่าที่เป็นไปได้สูงสุด (Maximum Likelihood Estimation: MLE)
หรือ กำหนดขึ้นเอง

$\ln$ = log ฐาน e

ตัวอย่างการคำนวณ

จากข้อมูลต่อไปนี้ จงแปลงค่าด้วยวิธี Box-Cox กำหนดให้ $\lambda = 1$

$x = [25, 27, 23, 35, 41, 32, 26, 22, 28, 34]$

วิธีทำ

จาก
$$\hat{x} = \frac{x^\lambda - 1}{\lambda}$$

$$\lambda = 1$$

เมื่อ $x = 25$ จะได้

$$\hat{x} = \frac{25^1 - 1}{1}$$

$$\hat{x} = 24$$

เมื่อ $x = 27$ จะได้

$$\hat{x} = \frac{27^1 - 1}{1}$$

$$\hat{x} = 26$$

.....

เมื่อ $x = 34$ จะได้

$$\hat{x} = \frac{34^1 - 1}{1}$$

$$\hat{x} = 33$$

5.5.2 โค้ดไพทอนสำหรับ Box-Cox

```
1 #Import libraries
2 import pandas as pd
3 from sklearn.preprocessing import PowerTransformer
4
5 #Load data
6 x = pd.read_csv('data/customer3.csv')
7
8 #Transform data
9 trans=PowerTransformer(method='box-cox', standardize=False)
10 trans.fit(x[['age']])
11 x[['age']] = trans.transform(x[['age']]).round(4)
12
13 print(f'Lambda : {trans.lambdas_[0]:.2f}')
14 x
```

✓ 0.0s

Lambda : -1.05

item	age	sex	education_level	occupation	total_spend
1	0.9223	หญิง	ปริญญาตรี	พนักงานขาย	4500
2	0.9249	ชาย	ปริญญาโท	พนักงานบัญชี	6200
3	0.9193	ชาย	ปริญญาตรี	พนักงานขาย	5000
4	0.9321	หญิง	ปริญญาเอก	พนักงานบุคคล	8000
5	0.9356	ชาย	ปริญญาโท	พนักงานบัญชี	120000
6	0.9298	หญิง	ปริญญาตรี	พนักงานบัญชี	3500
7	0.9236	หญิง	ปริญญาโท	พนักงานขาย	11000
8	0.9176	ชาย	ปริญญาตรี	พนักงานบุคคล	8750
9	0.9260	ชาย	ปริญญาตรี	พนักงานบุคคล	15000
10	0.9314	ชาย	ปริญญาตรี	พนักงานขาย	12000

หมายเหตุ : Scikit-learn จะประมาณค่า Lambda ให้อัตโนมัติ โดยใช้วิธี MLE

5.6 Discretization (Binning)

- เป็นเทคนิคการแปลงข้อมูลเชิงตัวเลข (Numerical) ให้เป็นข้อมูลเชิงกลุ่ม (Categorical) แบบ Ordinal โดยการแบ่งช่วงข้อมูลออกเป็น “ช่วงค่า เท่า ๆ กัน” หรือ “แบ่งตามจำนวนข้อมูล เท่า ๆ กัน”
- เหมาะกับ ML ที่ต้องการใช้ข้อมูลเชิงกลุ่ม (Category) เช่น Naïve Bayes และ Decision Tree
- ลดปัญหา Outliers
- ตัวอย่างเทคนิคของ Discretization เช่น
 - Equal-Width Binning (แบ่งช่วงค่าข้อมูลเท่า ๆ กัน)
 - Equal-Frequency Binning (แบ่งจำนวนแถวข้อมูลเท่า ๆ กัน)

Technique	ลด Outliers	ลด Noise	ข้อมูลสูญเสียรายละเอียด
Discretization	มาก	มาก	มาก
Box-Cox	ปานกลาง	ปานกลาง	น้อย

5.6.1 Equal-Width Binning

Equal-Width Binning เป็นวิธีการแบ่งช่วงข้อมูลเชิงตัวเลข (Numerical Variable) ออกเป็นหลายช่วง (Bins) ที่มีความกว้างเท่ากันทุกช่วง เพื่อช่วยลดความแปรปรวนของข้อมูล และลดผลกระทบจาก Outliers

หลักการ Equal-Width Binning

- กำหนดจำนวนช่วงข้อมูล (k) ที่ต้องการ
- คำนวณหาความกว้างของช่วง (w) จากสมการ
$$w = (max - min) / k$$
- กำหนดขอบเขตของช่วงข้อมูล ดังนี้
$$[min, min + 1 \cdot w), [min + 1 \cdot w, min + 2 \cdot w), \dots, [min + k-1 \cdot w, max]$$
- แทนค่า 0, 1, 2,..., $k-1$ ในแต่ละช่วง

ตัวอย่างการคำนวณ

จากข้อมูลต่อไปนี้ จงแปลงค่าด้วยวิธี Equal-Width Binning กำหนดให้มีทั้งหมด 3 Bins

$x = [25, 27, 23, 35, 41, 32, 26, 22, 28, 34]$

วิธีทำ

จาก $w = (max - min) / k$ โดยที่ $max = 41, min = 22, k = 3$

$[min, min + w), [min + w, min + 2w), [min + 2w, max] = 0, 1, 2$

จะได้

ความกว้างของช่วงข้อมูล คือ $w = (41 - 22) / 3 = 6.33, 2w = 12.66$

ช่วงข้อมูล คือ $[22, 22+6.33), [22+6.33, 22+12.66), [22+12.66, 41]$

$[22, 28.33), [28.33, 34.66), [34.66, 41]$

ข้อมูลที่ต้องการแปลง คือ $[25, 27, 23, 35, 41, 32, 26, 22, 28, 34]$

ข้อมูลที่แปลงแล้ว คือ $[0, 0, 0, 2, 2, 1, 0, 0, 0, 1]$

โค้ดไพทอนสำหรับ Equal-Width Binning

```
1 #Import libraries
2 import pandas as pd
3 from sklearn.preprocessing import KBinsDiscretizer
4
5 #Load data
6 x = pd.read_csv('data/customer3.csv')
7
8 #Discretize data with equal-width binning
9 trans=KBinsDiscretizer(n_bins=3, encode='ordinal',
10 | | | | | strategy='uniform')
11 trans.fit(x[['age']])
12 x[['age']]=trans.transform(x[['age']]).astype(int)
13 x
```

item	age	sex	education_level	occupation	total_spend
1	0	หญิง	ปริญญาตรี	พนักงานขาย	4500
2	0	ชาย	ปริญญาโท	พนักงานบัญชี	6200
3	0	ชาย	ปริญญาตรี	พนักงานขาย	5000
4	2	หญิง	ปริญญาเอก	พนักงานบุคคล	8000
5	2	ชาย	ปริญญาโท	พนักงานบัญชี	120000
6	1	หญิง	ปริญญาตรี	พนักงานบัญชี	3500
7	0	หญิง	ปริญญาโท	พนักงานขาย	11000
8	0	ชาย	ปริญญาตรี	พนักงานบุคคล	8750
9	0	ชาย	ปริญญาตรี	พนักงานบุคคล	15000
10	1	ชาย	ปริญญาตรี	พนักงานขาย	12000

5.6.2 Equal-Frequency Binning

Equal-Frequency Binning เป็นวิธีการแบ่งช่วงข้อมูลเชิงตัวเลข (Numerical Variable) ออกเป็นหลายช่วง (Bins) ที่มีจำนวนข้อมูลเท่ากันทุกช่วง เพื่อช่วยลดความแปรปรวนของข้อมูล และลดผลกระทบจาก Outliers

หลักการ Equal-Frequency Binning

- กำหนดจำนวนช่วงข้อมูล (k) ที่ต้องการ
- คำนวณหาจำนวนข้อมูล (m) ในแต่ละช่วง จากสมการ $m = \lfloor n/k \rfloor$ เมื่อ n คือจำนวนข้อมูลทั้งหมด
- แบ่งข้อมูลออกเป็น k ช่วง แต่ละช่วงมีข้อมูล m ตัว
- แทนค่า 0, 1, 2, ..., $k-1$ ในแต่ละช่วง โดยพิจารณาข้อมูลที่มีค่าน้อยไปหามาก
 - แทนค่า 0 ให้กับช่วงที่ 1
 - แทนค่า 1 ให้กับช่วงที่ 2
 - ทำแบบนี้ไปเรื่อย ๆ
 - หากแทนค่าจนครบจำนวน k ช่วงแล้ว ยังมีข้อมูลเหลืออยู่ ให้แทนค่าที่เหลือด้วยค่า $k-1$

ตัวอย่างการคำนวณ

จากข้อมูลต่อไปนี้ จงแปลงค่าด้วยวิธี Equal-Frequency Binning กำหนดให้มีทั้งหมด 3 Bins

$x = [25, 27, 23, 35, 41, 32, 26, 22, 28, 34]$

วิธีทำ

จาก $m = \lfloor n/k \rfloor$ โดยที่ $n = 10, k = 3$

จะได้

จำนวนข้อมูลในแต่ละช่วง คือ $m = \lfloor 10/3 \rfloor = 3$

ข้อมูลที่ต้องการแปลง คือ $[25, 27, 23, 35, 41, 32, 26, 22, 28, 34]$

ข้อมูลที่แปลงแล้ว คือ $[0, 1, 0, 2, 2, 2, 1, 0, 1, 2]$

3 ค่าแรก กำหนดให้เป็น 0

3 ค่าถัดไป กำหนดให้เป็น 1

3 ค่าถัดไปอีก กำหนดให้เป็น 2

ครบจำนวน k กลุ่มแล้ว เหลืออีก 1 ค่า คือ 41 กำหนดให้เป็น 2

โค้ดไพทอนสำหรับ Equal-Frequency Binning

```
1 #Import libraries
2 import pandas as pd
3 from sklearn.preprocessing import KBinsDiscretizer
4
5 #Load data
6 x = pd.read_csv('data/customer3.csv')
7
8 #Discretize data with equal-frequency binning
9 trans=KBinsDiscretizer(n_bins=3, encode='ordinal',
10 strategy='quantile',
11 quantile_method='linear')
12 trans.fit(x[['age']])
13 x[['age']]=trans.transform(x[['age']]).astype(int)
14 x
```

item	age	sex	education_level	occupation	total_spend
1	0	หญิง	ปริญญาตรี	พนักงานขาย	4500
2	1	ชาย	ปริญญาโท	พนักงานบัญชี	6200
3	0	ชาย	ปริญญาตรี	พนักงานขาย	5000
4	2	หญิง	ปริญญาเอก	พนักงานบุคคล	8000
5	2	ชาย	ปริญญาโท	พนักงานบัญชี	120000
6	2	หญิง	ปริญญาตรี	พนักงานบัญชี	3500
7	1	หญิง	ปริญญาโท	พนักงานขาย	11000
8	0	ชาย	ปริญญาตรี	พนักงานบุคคล	8750
9	1	ชาย	ปริญญาตรี	พนักงานบุคคล	15000
10	2	ชาย	ปริญญาตรี	พนักงานขาย	12000

หมายเหตุ : Scikit-learn จะแบ่งช่วงข้อมูลโดยใช้ค่า Quantile แทน $\lfloor n/k \rfloor$ ทำให้ผลการแปลงข้อมูลอาจไม่เท่ากันในบางกรณี

5.7 แบบฝึกหัด

จากข้อมูลที่กำหนดให้ จงทำการค้นหา Outliers ด้วยวิธีต่อไปนี้

a. วิธี Z-Score Outlier Detection สำหรับคอลัมน์ salary

b. วิธี IQR Outlier Detection สำหรับคอลัมน์ age

item	age	salary	department	score	city	education_level
1	25	32,000	Sales	78	Bangkok	Bachelor
2	28	35,000	IT	82	Chiangmai	Bachelor
3	31	40,000	HR	88	Bangkok	Master
4	29	36,500	Sales	75	Phuket	High School
5	45	120,000	IT	90	Bangkok	Master
6	22	28,000	HR	72	Bangkok	High School
7	27	34,000	Sales	79	Khon Kaen	Bachelor
8	30	38,000	IT	85	Bangkok	Master
9	102	33,500	HR	5	Chiangmai	High School
10	26	31,000	Sales	81	Bangkok	Bachelor
11	24	500,000	IT	77	Phuket	PhD
12	29	37,500	HR	84	Bangkok	Bachelor



THANK YOU FOR YOUR ATTENTION

 pichai_jo@rmutto.ac.th

